

Study Notes: Machine Learning, NLP & Deep Learning

A structured reference covering core concepts, algorithms, and terminology.

1. Machine Learning

1.1 What is Machine Learning?

Machine Learning (ML) is a branch of Artificial Intelligence that enables systems to **learn patterns from data** and improve performance on a task without being explicitly programmed with fixed rules. Instead of hard-coding logic, an ML model is trained on data and adjusts its internal parameters to minimize error on a given objective.

- **Traditional programming:** Input + Program → Output
- **Machine learning:** Input + Output (data) → Program (model)

1.2 Types of Machine Learning

1.2.1 Supervised Learning

The model learns from **labeled data** (input-output pairs). The goal is to learn a mapping function from inputs to outputs so it can predict outputs for new, unseen inputs.

- **Classification:** Output is a discrete category (e.g., spam vs not spam).
- **Regression:** Output is a continuous value (e.g., predicting house price).

1.2.2 Unsupervised Learning

The model works with **unlabeled data** and tries to find hidden structure or patterns on its own.

- **Clustering:** Grouping similar data points (e.g., customer segmentation).
- **Dimensionality Reduction:** Reducing number of features while preserving information (e.g., PCA).
- **Association Rule Learning:** Finding relationships between variables (e.g., market basket analysis).

1.2.3 Semi-Supervised Learning

Uses a **small amount of labeled data** combined with a **large amount of unlabeled data**. Useful when labeling data is expensive or time-consuming.

1.2.4 Reinforcement Learning (RL)

An **agent** learns to make decisions by interacting with an **environment**. The agent takes actions, receives **rewards or penalties**, and learns a policy that maximizes cumulative reward over time.

- **Agent:** The learner/decision maker.
- **Environment:** Everything the agent interacts with.
- **State:** Current situation of the agent.

- **Action:** Choice made by the agent.
- **Reward:** Feedback signal from the environment.
- **Policy:** Strategy the agent follows to choose actions.

1.3 Common Supervised Learning Algorithms

1.3.1 Linear Regression

Models the relationship between a dependent variable and one or more independent variables using a **straight line equation**: $y = b_0 + b_1x_1 + b_2x_2 + \dots + b_nx_n$. It minimizes the **sum of squared errors** between predicted and actual values.

1.3.2 Logistic Regression

Used for **binary classification**. It applies the **sigmoid function** to a linear combination of inputs to output a probability between 0 and 1.

1.3.3 Decision Trees

A tree-like model where each internal node represents a **test on a feature**, each branch represents an outcome, and each leaf represents a **class label or value**. Splits are chosen using criteria like **Gini Impurity** or **Information Gain (Entropy)**.

1.3.4 Random Forest

An **ensemble** of multiple decision trees trained on random subsets of data and features (bagging). Final prediction is made by majority vote (classification) or averaging (regression). Reduces overfitting compared to a single tree.

1.3.5 Support Vector Machine (SVM)

Finds the **optimal hyperplane** that separates classes with the **maximum margin**. Uses **kernel functions** (linear, polynomial, RBF) to handle non-linearly separable data by mapping it to higher dimensions.

1.3.6 K-Nearest Neighbors (KNN)

A **lazy, instance-based** algorithm. It classifies a new point based on the majority class among its **k nearest neighbors** in the feature space, typically using Euclidean distance.

1.3.7 Naive Bayes

A probabilistic classifier based on **Bayes' Theorem**, assuming features are **conditionally independent** given the class. Despite the naive assumption, it works well for text classification tasks like spam detection.

1.4 Common Unsupervised Learning Algorithms

1.4.1 K-Means Clustering

Partitions data into **k clusters** by iteratively assigning points to the nearest centroid and updating centroids until convergence. Requires specifying k in advance.

1.4.2 Hierarchical Clustering

Builds a tree of clusters (**dendrogram**) either by merging smaller clusters into larger ones (**agglomerative**) or splitting larger clusters into smaller ones (**divisive**).

1.4.3 DBSCAN

A **density-based** clustering method that groups points that are closely packed together and marks points in low-density regions as **outliers/noise**. Does not require specifying the number of clusters.

1.4.4 Principal Component Analysis (PCA)

A **dimensionality reduction** technique that transforms correlated features into a smaller set of **uncorrelated principal components** that capture maximum variance in the data.

1.5 Model Evaluation Metrics

1.5.1 Classification Metrics

- **Accuracy:** Proportion of correct predictions out of total predictions.
- **Precision:** Of all predicted positives, how many are actually positive ($TP / (TP + FP)$).
- **Recall (Sensitivity):** Of all actual positives, how many were correctly predicted ($TP / (TP + FN)$).
- **F1 Score:** Harmonic mean of precision and recall — useful for imbalanced datasets.
- **Confusion Matrix:** A table showing True Positives, True Negatives, False Positives, and False Negatives.
- **ROC-AUC:** Measures the model's ability to distinguish between classes across different thresholds.

1.5.2 Regression Metrics

- **MAE (Mean Absolute Error):** Average of absolute differences between predicted and actual values.
- **MSE (Mean Squared Error):** Average of squared differences — penalizes larger errors more heavily.
- **RMSE (Root Mean Squared Error):** Square root of MSE — same unit as the target variable.
- **R-squared (R2):** Proportion of variance in the target explained by the model.

1.6 Overfitting, Underfitting and the Bias-Variance Tradeoff

- **Overfitting:** Model learns training data too well, including noise, and performs poorly on new data. High variance, low bias.
- **Underfitting:** Model is too simple to capture the underlying pattern. High bias, low variance.
- **Bias-Variance Tradeoff:** Bias is error from overly simplistic assumptions; variance is error from sensitivity to small fluctuations in training data. The goal is to find a balance that minimizes total error.

1.7 Regularization

Regularization techniques add a penalty to the loss function to **discourage overly complex models** and reduce overfitting.

- **L1 Regularization (Lasso):** Adds the absolute value of coefficients as penalty; can shrink some coefficients to exactly zero (feature selection).
- **L2 Regularization (Ridge):** Adds the squared value of coefficients as penalty; shrinks coefficients but rarely to zero.
- **Elastic Net:** Combination of L1 and L2 regularization.

1.8 Cross-Validation

A technique to evaluate how well a model generalizes to independent data by splitting the dataset into multiple **folds**. In **k-fold cross-validation**, data is split into k parts; the model is trained on k-1 parts and tested on the remaining part, repeating k times and averaging the results.

1.9 Feature Engineering and Selection

- **Feature Engineering:** Creating new features from raw data to improve model performance (e.g., extracting day-of-week from a date).
- **Feature Scaling:** Normalization (min-max scaling) or Standardization (z-score) to bring features to a comparable range.
- **Feature Selection:** Choosing the most relevant features and removing redundant or irrelevant ones, using methods like correlation analysis, chi-square test, or recursive feature elimination.

1.10 Ensemble Methods

- **Bagging (Bootstrap Aggregating):** Trains multiple models independently on random subsets of data and combines predictions (e.g., Random Forest). Reduces variance.
- **Boosting:** Trains models sequentially, where each new model focuses on correcting errors made by previous models (e.g., AdaBoost, Gradient Boosting, XGBoost). Reduces bias.
- **Stacking:** Combines predictions from multiple different models using a meta-model that learns how to best combine them.

1.11 Typical Machine Learning Workflow

- 1. Problem definition and understanding the business objective.
- 2. Data collection.
- 3. Data cleaning and preprocessing (handling missing values, outliers, encoding).
- 4. Exploratory Data Analysis (EDA).
- 5. Feature engineering and selection.
- 6. Splitting data into training, validation, and test sets.
- 7. Model selection and training.
- 8. Model evaluation using appropriate metrics.
- 9. Hyperparameter tuning (Grid Search, Random Search, Bayesian Optimization).
- 10. Deployment and monitoring.

2. Natural Language Processing (NLP)

2.1 What is NLP?

Natural Language Processing is a field at the intersection of linguistics and AI that focuses on enabling computers to **understand, interpret, and generate human language** — both text and speech.

2.2 Text Preprocessing

- **Tokenization:** Splitting text into smaller units such as words or sentences.
- **Lowercasing:** Converting all text to lowercase for consistency.
- **Stopword Removal:** Removing common words (e.g., 'the', 'is', 'and') that carry little meaning.
- **Stemming:** Reducing words to their root form by chopping suffixes (e.g., 'running' → 'run'), often crude and rule-based (Porter Stemmer).
- **Lemmaization:** Reducing words to their dictionary base form using vocabulary and grammar rules (e.g., 'better' → 'good'), more accurate than stemming.
- **Removing Punctuation/Special Characters:** Cleaning noise from raw text.

2.3 Text Representation Techniques

2.3.1 Bag of Words (BoW)

Represents text as a vector of **word counts or frequencies**, ignoring grammar and word order. Simple but results in large, sparse vectors and loses context.

2.3.2 TF-IDF (Term Frequency - Inverse Document Frequency)

Weights words by how frequently they appear in a document (**TF**) relative to how common they are across all documents (**IDF**). Words common in one document but rare overall get higher weight, highlighting distinctive terms.

2.3.3 N-grams

Contiguous sequences of **n** words used to capture some local word order and context (e.g., bigrams = 2 words, trigrams = 3 words).

2.3.4 Word Embeddings

Dense, low-dimensional vector representations of words that capture **semantic meaning**, such that similar words have similar vectors.

- **Word2Vec:** Learns embeddings using either the CBOW (predicting a word from context) or Skip-gram (predicting context from a word) architecture.
- **GloVe:** Learns embeddings based on global word co-occurrence statistics across the entire corpus.
- **FastText:** Extends Word2Vec by representing words as bags of character n-grams, handling out-of-vocabulary and rare words better.
- **Contextual Embeddings (e.g., BERT, ELMo):** Generate different vectors for the same word depending on its context in a sentence.

2.4 Core NLP Tasks

- **Part-of-Speech (POS) Tagging:** Assigning grammatical categories (noun, verb, adjective, etc.) to each word.
- **Named Entity Recognition (NER):** Identifying and classifying entities like names of people, organizations, locations, and dates in text.

- **Parsing:** Analyzing grammatical structure of a sentence (constituency parsing or dependency parsing).
- **Sentiment Analysis:** Determining the emotional tone (positive, negative, neutral) of a piece of text.
- **Text Classification:** Assigning predefined categories to text documents (e.g., spam detection, topic labeling).
- **Machine Translation:** Automatically translating text from one language to another.
- **Text Summarization:** Producing a shorter version of a text while retaining key information (extractive or abstractive).
- **Question Answering:** Automatically answering questions posed in natural language.
- **Coreference Resolution:** Determining which words refer to the same entity (e.g., 'he' referring to a previously mentioned name).

2.5 Language Models

A language model estimates the **probability distribution over sequences of words**, essentially predicting the likelihood of the next word given previous words.

- **N-gram Language Models:** Estimate probability of a word based on the previous n-1 words using frequency counts.
- **Neural Language Models:** Use neural networks (RNN, LSTM, or Transformer-based) to model word sequences, capturing longer-range dependencies.

2.6 Sequence Models for NLP

- **Recurrent Neural Networks (RNN):** Process sequences one element at a time while maintaining a hidden state to capture information from previous steps. Suffer from vanishing/exploding gradients on long sequences.
- **LSTM (Long Short-Term Memory):** A type of RNN with gating mechanisms (input, forget, output gates) that allows it to retain information over longer sequences.
- **GRU (Gated Recurrent Unit):** A simplified version of LSTM with fewer gates (reset and update gates), computationally cheaper while achieving similar performance.

2.7 Attention Mechanism and Transformers

Attention allows a model to focus on the most relevant parts of the input sequence when producing each part of the output, rather than compressing all information into a single fixed vector.

The **Transformer architecture** (introduced in 'Attention Is All You Need') relies entirely on **self-attention** mechanisms instead of recurrence, allowing **parallel processing** of sequences and better handling of long-range dependencies. Key components include multi-head self-attention, positional encoding, and feed-forward layers.

2.8 Pretrained Language Models

- **BERT (Bidirectional Encoder Representations from Transformers):** Reads text bidirectionally and is pretrained using masked language modeling; strong for understanding tasks (classification, NER, QA).

- **GPT (Generative Pretrained Transformer):** A decoder-only, autoregressive transformer trained to predict the next word; strong for text generation.
- **T5 (Text-to-Text Transfer Transformer):** Frames every NLP task as a text-to-text problem.
- **Transfer Learning in NLP:** Pretraining a large model on massive text corpora, then fine-tuning it on a smaller, task-specific dataset.

2.9 NLP Evaluation Metrics

- **BLEU Score:** Measures overlap of n-grams between generated and reference text, commonly used in machine translation.
- **ROUGE Score:** Measures overlap of n-grams, mainly used for evaluating summarization.
- **Perplexity:** Measures how well a language model predicts a sample; lower perplexity indicates a better model.
- **F1 Score:** Used for tasks like NER and QA to balance precision and recall.

3. Deep Learning

3.1 What is Deep Learning?

Deep Learning is a subset of Machine Learning based on **artificial neural networks** with multiple layers ('deep' architectures). Unlike traditional ML, which often requires manual feature engineering, deep learning models can **automatically learn hierarchical feature representations** directly from raw data.

3.2 Neural Network Basics

- **Neuron/Perceptron:** The basic computational unit; computes a weighted sum of inputs plus a bias, then applies an activation function.
- **Layers:** Neurons are organized into an **input layer**, one or more **hidden layers**, and an **output layer**.
- **Weights and Biases:** Learnable parameters adjusted during training.
- **Fully Connected (Dense) Layer:** Every neuron connects to every neuron in the next layer.

3.3 Activation Functions

- **Sigmoid:** Squashes input to a range between 0 and 1; suffers from vanishing gradients for large inputs.
- **Tanh:** Squashes input to a range between -1 and 1; zero-centered but still suffers from vanishing gradients.
- **ReLU (Rectified Linear Unit):** Outputs the input directly if positive, otherwise zero. Computationally efficient and reduces vanishing gradient issues, but can suffer from 'dying ReLU' problem.
- **Leaky ReLU:** Allows a small, non-zero gradient when the input is negative, addressing the dying ReLU problem.

- **Softmax:** Converts a vector of raw scores into a probability distribution, commonly used in the output layer for multi-class classification.

3.4 Forward and Backward Propagation

- **Forward Propagation:** Input data passes through the network layer by layer to produce an output/prediction.
- **Loss Function:** Measures the difference between predicted output and actual target (e.g., Cross-Entropy for classification, MSE for regression).
- **Backpropagation:** Computes the gradient of the loss with respect to each weight by applying the chain rule, propagating error backward from output to input layers.
- **Gradient Descent:** An optimization algorithm that updates weights in the direction that reduces the loss, scaled by a learning rate.

3.5 Optimizers

- **SGD (Stochastic Gradient Descent):** Updates weights using the gradient computed from a single sample or small batch, rather than the full dataset.
- **Momentum:** Adds a fraction of the previous update to the current one, helping accelerate convergence and reduce oscillation.
- **RMSprop:** Adapts the learning rate for each parameter based on a moving average of squared gradients.
- **Adam (Adaptive Moment Estimation):** Combines momentum and RMSprop ideas; maintains adaptive learning rates for each parameter. Widely used as a default optimizer.

3.6 Regularization in Deep Learning

- **Dropout:** Randomly deactivates a fraction of neurons during training to prevent co-adaptation and reduce overfitting.
- **Batch Normalization:** Normalizes the inputs of each layer to stabilize and speed up training.
- **Early Stopping:** Halts training when validation performance stops improving, preventing overfitting.
- **Data Augmentation:** Artificially expanding the training set by applying transformations (rotation, flipping, cropping for images) to improve generalization.
- **Weight Decay:** Adds an L2 penalty on weights during optimization, similar to Ridge regularization.

3.7 Convolutional Neural Networks (CNNs)

CNNs are specialized for processing **grid-like data** such as images.

- **Convolutional Layer:** Applies learnable filters/kernels that slide over the input to detect local patterns like edges, textures, and shapes.
- **Pooling Layer:** Reduces spatial dimensions (e.g., Max Pooling, Average Pooling) while retaining important information, reducing computation and overfitting.
- **Flatten Layer:** Converts pooled feature maps into a 1D vector before feeding into fully connected layers.

- **Notable Architectures:** LeNet, AlexNet, VGG, GoogLeNet (Inception), ResNet (introduces skip/residual connections to allow training very deep networks).
- **Applications:** Image classification, object detection, image segmentation, facial recognition.

3.8 Recurrent Architectures (Recap in Deep Learning Context)

RNNs, LSTMs, and GRUs (covered in Section 2.6) are also core deep learning architectures used for **sequential data** such as time series, text, and speech, where order and temporal dependency matter.

3.9 Autoencoders

An unsupervised neural network trained to **reconstruct its input**. It consists of an **encoder** that compresses input into a lower-dimensional latent representation, and a **decoder** that reconstructs the original input from that representation. Used for dimensionality reduction, denoising, and anomaly detection.

- **Variational Autoencoder (VAE):** A probabilistic version that learns a distribution over the latent space, enabling generation of new, similar data samples.

3.10 Generative Adversarial Networks (GANs)

Consist of two competing networks trained simultaneously:

- **Generator:** Creates fake data samples trying to resemble real data.
- **Discriminator:** Tries to distinguish between real and generated (fake) data.

The two networks are trained in a **minimax game** — the generator improves at fooling the discriminator, while the discriminator improves at detecting fakes, eventually producing highly realistic synthetic data (images, audio, etc.).

3.11 Transformer Architecture (Deep Learning View)

Transformers (see Section 2.7) are now the dominant deep learning architecture across both NLP and increasingly computer vision (Vision Transformers). Core building blocks include **self-attention**, **multi-head attention**, **positional encodings**, and **residual connections with layer normalization**.

3.12 Transfer Learning

Reusing a model that has been **pretrained on a large dataset** (e.g., ImageNet, large text corpora) and adapting it to a new, often smaller, task-specific dataset. Common strategies include **feature extraction** (freezing pretrained layers) and **fine-tuning** (updating some or all pretrained weights on the new task).

3.13 Hyperparameter Tuning in Deep Learning

- **Learning Rate:** Controls the step size of weight updates; too high can cause divergence, too low can cause slow convergence.
- **Batch Size:** Number of samples processed before updating model weights.
- **Number of Epochs:** Number of complete passes through the training dataset.
- **Number of Layers/Neurons:** Determines model capacity and complexity.

- **Tuning Methods:** Grid Search, Random Search, Bayesian Optimization, and learning rate schedulers.

3.14 Common Deep Learning Frameworks

- **TensorFlow:** An open-source framework developed by Google, widely used in production environments; includes the high-level Keras API.
- **PyTorch:** An open-source framework developed by Meta, popular in research for its dynamic computation graph and intuitive syntax.

3.15 Machine Learning vs Deep Learning vs NLP — How They Relate

- **Machine Learning** is the broad umbrella field covering algorithms that learn from data.
- **Deep Learning** is a subfield of ML using multi-layered neural networks, often requiring large data and computational power, but reducing the need for manual feature engineering.
- **NLP** is an application domain (working with human language) that uses both traditional ML techniques (e.g., Naive Bayes, TF-IDF) and modern Deep Learning techniques (e.g., RNNs, Transformers) to solve language-related problems.